

DataPilot

Technical Documentation

Describe the data you need in plain English. Get a clean, sourced dataset.

Item	Value
Live application	https://datapilot-gamma.vercel.app
Source code	https://github.com/chetanprasadoofficial/datapilot
Document date	30 September 2026
Stack	Next.js (App Router, TypeScript), Tailwind CSS, Supabase (Postgres), Google Gemini API, Vercel
License	MIT

Contents

- 1. Overview and problem statement
- 2. Features
- 3. System architecture
- 4. The planner
- 5. Data connectors (permitted sources)
- 6. The five-step pipeline
- 7. Data model
- 8. HTTP API
- 9. User interface
- 10. Setup and configuration
- 11. Deployment
- 12. Test results and engineering notes
- 13. Security and compliance
- 14. Limitations
- 15. Roadmap
- 16. License
- Appendix A. Full SQL schema
- Appendix B. Example plan

1. Overview and problem statement

DataPilot turns a natural-language request into a data-collection workflow, runs that workflow against permitted public sources, cleans and deduplicates the results, and shows every row together with where it came from. Rows that are dropped are not silently discarded: each one is stored with the reason it was rejected.

The problem. Collecting data from the web usually means writing a one-off script for every source, then cleaning, deduplicating and checking the output by hand. The end result is often a spreadsheet that nobody can trace back to its origin, and a process that cannot be repeated or compared over time.

The approach. DataPilot makes the whole workflow visible and repeatable. A request becomes a reviewable plan. The plan runs as a fixed pipeline with live status. Every result row carries its sources. Every run is stored so it can be opened again, re-run, and compared with the previous run.

2. Features

- **Prompt to plan.** An LLM (Google Gemini) converts a request into a structured plan: entity, fields, filters, sources and row limit. The plan is shown for review before anything runs, and individual sources can be switched off.
- **Five-step pipeline.** collect, normalize, validate, dedupe, rank, with live per-step status and item counts.
- **Honest filtering.** Every rejected row is stored with a reason and is visible in the Rejected tab.

- **Cross-source deduplication.** Rows are merged on canonical URL and on title plus company. A merged row keeps every source it came from.
- **Source inspector.** Clicking a row shows connector, source URL, fetch time, robots.txt status and evidence.
- **Results table.** Search, source filter and column sorting.
- **Export.** CSV and JSON downloads.
- **History and re-run.** Every run is stored. Re-running a workflow shows the change in final row count against the previous run of the same workflow.
- **Reliable planner.** If the LLM is unavailable, a rule-based template plan is used and the plan page says so. Constraints stated in the prompt (for example the number of days, and the word remote) are enforced in code after planning.

3. System architecture

The application is a single Next.js project. Pages and API routes are deployed together on Vercel. All state is stored in a Supabase Postgres database. The planner calls the Gemini API. The connectors call public HTTP APIs.



Layer	Technology	Role
Front end	Next.js App Router, React, Tailwind CSS	Prompt page, plan review, live run page, history
Back end	Next.js route handlers (TypeScript)	Planning, run execution, records, export
Database	Supabase (PostgreSQL)	Workflows, runs, steps, records, provenance, rejected rows
LLM	Google Gemini API	Turns a prompt into a JSON plan
Validation	Zod	Checks the plan structure returned by the LLM
Hosting	Vercel	Serverless hosting with automatic deploys from the main branch

4. The planner

The planner lives in **lib/planner.ts**. It takes the user's prompt and returns a plan object that describes the whole workflow.

4.1 Plan structure

Field	Type	Meaning
goal	string (min 5 characters)	Short statement of the request
entity	string	For example job_posting or news_post
fields	array of 2 to 12	Columns of the output. Each has a name, a type (string, url, date, number, string[]) and a required flag
filters	object	keywords, location, max_age_days and similar constraints
sources	array of 1 to 4	Connector name (remotive, arbeitnow, hn_algolia, web_page) and its parameters
steps	array	Defaults to collect, normalize, validate, dedupe, rank
dedupe_keys	array of strings	Defaults to url
limit	number from 10 to 50	Maximum number of rows
assumptions	array of strings (optional)	Assumptions the planner made, shown on the plan page

4.2 Calling Gemini

- The request asks for a JSON response and uses temperature 0.2.
- Two models are configured through environment variables: GEMINI_MODEL is tried first and GEMINI_FALLBACK_MODEL second.
- On HTTP 503 (temporary overload) the call waits and retries, up to three attempts per model. On 404, 429 or 400 it moves to the next model immediately.
- Markdown fences around the JSON are stripped before parsing.
- The planner makes up to two attempts. If the first answer fails schema validation, the validation error is sent back to the model with a request to fix it.

4.3 Template fallback

If the LLM cannot be reached or does not return a valid plan, a rule-based template plan is built from the prompt. The plan page then shows the assumption "Template plan used because the AI planner was unavailable". A message beginning PLANNER FALLBACK is also written to the server log with the underlying reason.

- Prompts containing words such as funding, startup, hacker news, launch or news produce a news_post plan using the hn_algolia connector.
- All other prompts produce a job_posting plan with the fields title, company, location, url, posted_at and tags, using remotive, arbeitnow and hn_algolia, with a default age limit of 7 days and a limit of 50 rows.
- Keywords are taken from the prompt after removing common filler words such as find, list, collect, recent, posted, last, days, weeks, from, with, that, job, jobs and remote.

4.4 Constraint enforcement

After either the LLM plan or the template plan is produced, an **enforce** step overrides values that the prompt states explicitly. This guarantees that the plan matches what was asked, whichever path produced it.

- A phrase such as "last 30 days", "past 2 weeks" or "previous 3 months" sets `max_age_days` (a week counts as 7 days and a month as 30 days).
- For `job_posting` plans, the word `remote` sets the location filter to `remote`. If the prompt also mentions Delhi, NCR, Noida, Gurgaon or Gurugram, the location filter becomes "Remote, Delhi-NCR".
- The `days` parameter of any `hn_algia` source is set to `max_age_days`.
- For `job_posting` plans, `hn_algia` is removed unless the prompt mentions Hacker News or HN, because its job coverage is very low. It is only removed if at least one other source remains.

5. Data connectors (permitted sources)

The connectors live in **lib/connectors.ts**. Each one returns raw items that include the connector name, the raw source object, the source URL, the fetch time and a short snippet. DataPilot only collects from sources that publish a public API or feed intended for programmatic access.

Connector	Source	Behaviour
remote	Remote public jobs API	If the search text contains develop, engineer, software, programm, intern or sde, the category software-dev endpoint (limit 100) is used. Otherwise a text search (limit 50) is used. Up to 100 items are returned. The public API returned 16 items per query in testing.
arbeitnow	Arbeitnow job board API	Fetches pages 1 to 3, then filters locally: a job is kept if any search term longer than 2 characters appears in its title, tags or description. Up to 150 items are returned. The API is European, so many jobs are on-site in Germany.
hn_algia	Hacker News search by date (Algolia)	Searches stories created within the last N days (default 7) for the query text, 40 hits per page. Best for news and launch prompts.
web_page	Arbitrary public web pages	Accepted by the plan schema but not enabled. See Limitations.

5.1 Request behaviour

- Every request carries the user agent `DataPilotBot/1.0` with the project URL.
- Every request has an 8 second timeout and one retry for transient errors, with a short pause.
- On HTTP 403 or 429 the connector stops immediately and does not retry, so a rate-limited source is not hammered.
- A connector failure does not fail the whole run. The run continues with the other sources and is marked partial, and the run page names the failed source.

6. The five-step pipeline

The pipeline is executed by **lib/executor.ts** and implemented in **lib/transforms.ts**. Each step writes its status, item counts and details to the `run_steps` table, and the run page polls for progress every 1.5 seconds.

6.1 Collect

All enabled sources are called. The user can switch sources off on the plan page, and the switched-off sources are passed as overrides when the run starts. Per-source item counts and timings are stored in the step details and shown on the run page.

6.2 Normalize

- A mapper per connector converts the raw source object into a canonical record with title, company, location, url, posted_at, tags, points, author and text.
- Remote locations are written as Remote (required region). Arbeitnow locations combine the word Remote, when the job's remote flag is true, with the listed location.
- Field names chosen by the LLM are mapped to canonical keys through an alias table, for example posted_date, published_at and created_at map to posted_at, and link and source_url map to url.
- URLs are canonicalized: the fragment is removed, tracking parameters (utm_ prefixes, ref, fbclid, gclid) are removed, and a trailing slash is removed.
- Dates are converted to ISO format. Numbers, dates, URLs and string arrays are coerced to the type declared in the plan.
- A completeness value is computed as the share of planned fields that are filled.

6.3 Validate

Each record is checked against the following rules in this order. The first rule that fails becomes the stored rejection reason.

Order	Rule	Rejection reason
1	Every required field in the plan must be present	missing field-name
2	The URL must start with http or https	invalid url
3	If max_age_days is set and the record has a date, it must not be older than the limit	older than N days
4	If a location filter is set, the record must have a location and it must match. Delhi or NCR in the filter also matches Delhi, Noida, Gurgaon, Gurugram, Ghaziabad, Faridabad and NCR	location does not match
5	For job_posting plans, titles containing sales, network, service desk, helpdesk, support, customer, hardware, mechanical, civil, electrical, security or data scientist are excluded	not a software role
6	At least one keyword must appear in the title, company, tags or text	no keyword match

Keyword matching. Plan keywords are split into words. Filler words (software, senior, junior, job, jobs, the, and, for, in, of, remote, posted, recent, latest, hiring, role, roles, position, positions) are ignored. The words developer, engineer and programmer are treated as one group, so a title such as Senior Backend Engineer matches a request for software developer jobs. Matching uses word boundaries with an optional s, ship or ing ending, so intern matches internship but does not match international.

6.4 Deduplicate

Records are merged when they share a canonical URL, or when they share the same title and company after both are lower-cased and stripped of punctuation. When two records are merged, the one with the higher completeness is kept and the source list of both is combined. The number of merged records is shown as Duplicates merged.

6.5 Rank

Each remaining record receives a score, and the records are sorted from highest to lowest score.

```
score = round( (keyword_hits * 2 + freshness + completeness) * 100 ) / 100

freshness    = max(0, 1 - age_in_days / max_age_days)    (max_age_days defaults to 30)
keyword_hits = number of plan keywords found in the record
completeness = filled planned fields / all planned fields
```

7. Data model

The database has seven tables. The full SQL is in Appendix A and in the repository file `supabase/schema.sql`. Deleting a workflow removes its runs and everything that belongs to them.

Table	Purpose	Key columns
workflows	One row per plan created from a prompt	id, prompt, plan (jsonb), plan_raw, created_at
runs	One row per execution of a workflow	id, workflow_id, status, stats (jsonb), error, started_at, finished_at
run_steps	Status of each pipeline step in a run	run_id, name, position, status, items_in, items_out, detail (jsonb)
sources	Every fetched source item, for provenance	run_id, connector, url, fetched_at, http_status, robots_allowed, snippet
records	Final cleaned rows	run_id, data (jsonb), dedupe_key, score, completeness, grounded
record_sources	Links a record to every source it came from	record_id, source_id, evidence
rejected_rows	Rows dropped during validation	run_id, raw (jsonb), reason

Run status values seen in the application are queued, running, done, partial, failed and cancelled. The stats object holds collected, rejected, duplicates, final and a map of rejection reasons with their counts.

8. HTTP API

Method and path	Purpose
POST /api/plan	Creates a plan from a prompt (body: prompt) and returns the workflow id
GET /api/workflows?id=ID	Returns one workflow with its plan
POST /api/runs	Starts a run (body: workflowId, optional overrides.disabledSources) and returns runId
GET /api/runs	Lists runs with their workflow, newest first, for the history page
GET /api/runs/ID	Returns the run, its steps and its workflow; polled while the run is active
GET /api/runs/ID/records	Returns the final rows with their sources
GET /api/runs/ID/rejected	Returns rejected rows with reasons
GET /api/runs/ID/export?format=csv json	Downloads the results as CSV or JSON

9. User interface

Page	What it does
/	Prompt box, Create plan button, three example prompts and a link to History
/plan/ID	Shows goal, entity, row limit, fields, filters, sources with on/off checkboxes and assumptions, then a Run workflow button
/runs/ID	Status badge, four summary cards (Collected, Rejected, Duplicates merged, Final rows), step progress, per-source timings and errors, rejection reasons, Results and Rejected tabs, search, source filter, sortable columns, source inspector, Export CSV, Export JSON and Re-run
/history	Every run with its goal, time, sources, status, final rows, collected, rejected and the change against the previous run of the same workflow, with Open and Re-run buttons

Source inspector

Clicking a result row opens a side panel that lists every field value and, for each source of that row, the connector name, the source URL, the fetch time, the robots.txt status and the evidence text if one is stored. For public API sources the robots.txt status is shown as not applicable.

10. Setup and configuration

10.1 Requirements

- Node.js and npm
- A Supabase project
- A Google Gemini API key

10.2 Install

```
git clone https://github.com/chetanprasadofficial/datapilot.git
cd datapilot
npm install
```

10.3 Environment variables

Create a file named `.env.local` in the project root with the following values.

Variable	Meaning
GEMINI_API_KEY	Gemini API key
GEMINI_MODEL	Primary model name, for example <code>gemini-flash-latest</code>
GEMINI_FALLBACK_MODEL	Second model tried if the first fails, for example <code>gemini-3-flash-preview</code>
SUPABASE_URL	Project URL of the Supabase project
SUPABASE_SERVICE_ROLE_KEY	Service role key. Used only on the server and never sent to the browser

10.4 Create the tables

Open the Supabase dashboard, go to the SQL Editor, paste the contents of `supabase/schema.sql` and run it.

10.5 Run

```
npm run dev
```

Open `http://localhost:3000`.

11. Deployment

- The project is connected to a Vercel project and deploys automatically on every push to the main branch.
- The five environment variables above are set in Vercel under Settings, Environment Variables, for Production and Preview.
- Vercel applies new environment variables only to new deployments, so a redeploy is needed after changing them.
- The stable production address is `https://datapilot-gamma.vercel.app`. Each deployment also has its own unique address, which is a snapshot of that one deployment and may be removed by Vercel's retention policy.
- The production address opens without a Vercel login, which was verified in a private browser window.

12. Test results and engineering notes

12.1 Observed runs

Prompt	Collected	Rejected	Final rows	Notes
Remote or Delhi-NCR software internships, last 7 days	66	65 to 66	0 to 1	Internships are rare in free public feeds, so few rows pass. The Rejected tab shows the reasons.
Remote software developer jobs, last 14 days (first version of the filters)	67	66	1	Exposed an over-strict keyword filter
Remote software developer jobs, last 30 days (after keyword fix)	168	145	22	Some unrelated engineer roles and rows with no location still passed
Remote software developer jobs, last 30 days (after role and location fixes, planner enforcement)	166	155	11	All rows remote and inside the 30-day window. Reproduced on the live site

In the final run the rejection reasons were: location does not match (132), no keyword match (15), not a software role (7) and older than 30 days (1). Re-running the same workflow on the live site produced identical counts, and the History page showed +0 compared with the previous run.

12.2 Problems found and how they were fixed

Problem	Cause	Fix
Only 1 row passed for a broad jobs prompt	Keywords were matched as an exact phrase, so Senior Backend Engineer failed a software developer request	Keywords are split into words, filler words are dropped and developer, engineer and programmer are treated as a group
Short keywords matched inside longer words	Plain substring matching	Word-boundary matching with optional s, ship or ing ending
Rows with no location passed a remote filter	The location check only ran when a location existed	A missing location now fails the location check
Unrelated engineer roles passed	engineer was expanded to the whole role group	Exclusion list for non-software titles
Plan ignored the stated time window and remote requirement	The Gemini call failed with HTTP 429 (free-tier quota), and the fallback template used fixed values	Constraints are now enforced in code after every plan, and the fallback reason is logged
Volume was low	Small source responses	Arbeittnow fetches three pages, and Remoteive uses its category endpoint

13. Security and compliance

- Only public API sources are used. No login-only or scraping-forbidden sites are accessed.
- Requests identify themselves with a DataPilotBot user agent.
- Requests have timeouts, retry once for transient errors, and stop on HTTP 403 and 429.

- The Supabase service role key is used only in server code. It must never be exposed to the browser or committed to the repository.
- Source terms apply. The Remote API in particular asks users not to fetch too often.
- Every result row keeps its source URL, fetch time and connector name, so any value can be traced.

14. Limitations

- **Free public APIs are thin.** Volume depends on what the sources expose. Internship prompts often end with zero or one row.
- **Remote returns a small number of items** per request through its public API.
- **Hacker News is a poor source for job prompts.** Job postings there are comments inside a monthly thread, not stories.
- **Role filtering is heuristic.** Title keywords and an exclusion list decide what counts as a software role, so edge cases can be misclassified.
- **Location matching is text based.** Rows with a blank location are rejected when a location filter is set.
- **LLM quota.** The planner uses the Gemini free tier. When the quota is exhausted, the template plan is used and the plan page says so.
- **The web_page connector is not implemented.** Extraction from arbitrary public pages, with a robots.txt check, per-page rate limiting and verification that extracted values appear in the page text, is designed and accepted by the plan schema but is not enabled.
- **Serverless limits.** Very large runs are out of scope because serverless functions have execution time limits.

15. Roadmap

- web_page connector with robots.txt enforcement and evidence-grounded extraction.
- More API connectors, for example events, papers and grants.
- Scheduled runs and change alerts on re-run.
- Editable filters on the plan page, in addition to the source toggles.

16. License

DataPilot is released under the MIT License. The full text is in the LICENSE file in the repository.

Appendix A. Full SQL schema

```

create table workflows (
  id uuid primary key default gen_random_uuid(),
  prompt text not null,
  plan jsonb not null,
  plan_raw text,
  created_at timestamptz default now()
);

create table runs (
  id uuid primary key default gen_random_uuid(),
  workflow_id uuid references workflows(id) on delete cascade,
  status text not null default 'queued',
  stats jsonb default '{}',
  error text,
  started_at timestamptz,
  finished_at timestamptz,
  created_at timestamptz default now()
);

create table run_steps (
  id uuid primary key default gen_random_uuid(),
  run_id uuid references runs(id) on delete cascade,
  name text not null,
  position int not null,
  status text not null default 'pending',
  items_in int default 0,
  items_out int default 0,
  detail jsonb default '{}',
  started_at timestamptz,
  finished_at timestamptz
);

create table sources (
  id uuid primary key default gen_random_uuid(),
  run_id uuid references runs(id) on delete cascade,
  connector text not null,
  url text not null,
  fetched_at timestamptz default now(),
  http_status int,
  robots_allowed boolean,
  snippet text
);

create table records (
  id uuid primary key default gen_random_uuid(),
  run_id uuid references runs(id) on delete cascade,
  data jsonb not null,
  dedupe_key text,
  score real,
  completeness real,
  grounded boolean default true,
  created_at timestamptz default now()
);

create table record_sources (
  record_id uuid references records(id) on delete cascade,
  source_id uuid references sources(id) on delete cascade,
  evidence text,
  primary key (record_id, source_id)
);

create table rejected_rows (
  id uuid primary key default gen_random_uuid(),
  run_id uuid references runs(id) on delete cascade,
  raw jsonb,
  reason text
);

```

Appendix B. Example plan

This is the shape of the plan produced for the prompt "Find remote software developer jobs posted in the last 30 days".

```
{
  "goal": "Find remote software developer jobs posted in the last 30 days",
  "entity": "job_posting",
  "fields": [
    { "name": "title",      "type": "string", "required": true },
    { "name": "company",   "type": "string", "required": true },
    { "name": "location",  "type": "string", "required": false },
    { "name": "url",       "type": "url",    "required": true },
    { "name": "posted_at", "type": "date",   "required": false },
    { "name": "tags",      "type": "string[]", "required": false }
  ],
  "filters": {
    "keywords": ["software", "developer"],
    "location": "remote",
    "max_age_days": 30
  },
  "sources": [
    { "connector": "remotive", "params": { "search": "software developer" } },
    { "connector": "arbeitnow", "params": { "search": "software developer" } }
  ],
  "steps": ["collect", "normalize", "validate", "dedupe", "rank"],
  "dedupe_keys": ["url"],
  "limit": 50
}
```